



SMART RESTAURANT ORDERING & KITCHEN DISPLAY SYSTEM

PROJECT REPORT

Database Systems Engineering and Distributed Backend Development

Team - 25 | Section - 12 | CSIT

Guide - Dr. Khajashareef

Department of Computer Science and Information Technology

KLH University

Hitesh Suthar - 2520090078

V. Nikith Chowdary - 2520090219

B. Vasudeva - 2520090136

DECLARATION

The project report entitled “SMART RESTAURANT ORDERING & KITCHEN DISPLAY SYSTEM” records the work carried out by Team 25, Section 12, CSIT as part of DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT.

- Hitesh Suthar - 2520090078
- V. Nikith Chowdary - 2520090219
- B. Vasudeva - 2520090136

The report documents the system design, implementation, database models, security controls, real-time communication, testing considerations, and deployment approach of the project under the guidance of Dr. Khajashareef.

Hitesh Suthar - 2520090078

V. Nikith Chowdary - 2520090219

B. Vasudeva - 2520090136

CERTIFICATE

This is to certify that the project report entitled “SMART RESTAURANT ORDERING & KITCHEN DISPLAY SYSTEM” has been prepared and submitted by Team 25, Section 12, CSIT in partial fulfillment of the requirements of DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT at KLH University under the guidance of Dr. Khajashareef.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We sincerely thank Dr. Khajashareef for guidance, encouragement, and technical direction during the development of this project.

We express our gratitude to the Department of Computer Science and Information Technology and KLH University for the academic support and facilities provided. We also thank our teaching and non-teaching staff, classmates, friends, and family members for their support.

ABSTRACT

The Smart Restaurant Ordering & Kitchen Display System is a web-based application that digitizes restaurant ordering and kitchen coordination. Customers scan a table-specific QR code, access a digital menu, select food items and quantities, build a cart, and place an order from their table.

The backend validates the table and menu items, recalculates prices on the server, applies 5% tax, stores the order in MongoDB, and publishes a real-time new-order event to the kitchen. Kitchen staff manage orders through PENDING → PREPARING → READY → DELIVERED, while customers can follow status changes through Socket.IO.

Managers receive a dashboard for today's revenue, orders, active orders, completed orders, menu/category management, and table/QR management. The system uses React, Vite, Node.js, Express.js, MongoDB/Mongoose, Socket.IO, JWT, Argon2, Helmet, CORS, rate limiting, and QRCode React.

TABLE OF CONTENTS

Chapter	Topic	Page
1	Introduction	6
2	System Requirements	8
3	Technology Stack	9
4	System Architecture	10
5	Database Design	12
6	Implementation	14
7	Features	16
8	Testing & Validation	18
9	Deployment	20
10	Challenges & Limitations	21
11	Future Enhancements	22
12	Conclusion	23
13	References	24
14	Appendices	25

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Traditional restaurant ordering depends on communication between customers, waiters, and kitchen staff. Manual ordering can cause incorrect orders, delays in processing, difficulty in tracking orders, and challenges in coordinating restaurant operations.

The proposed system replaces the manual workflow with a table-specific QR ordering process. A customer scans the QR code, the system validates the table, and the digital menu becomes available for direct ordering.

1.2 Problem Statement

The project addresses the need for a reliable digital flow that connects table identification, menu availability, order creation, kitchen processing, order tracking, and manager visibility.

1.3 Scope

- QR-based table validation and digital menu access.
- Customer cart, checkout and order tracking.
- Chef dashboard for real-time order reception and status changes.
- Manager dashboard for menu, categories, tables, QR codes and KPIs.
- REST APIs, MongoDB persistence, JWT authentication and Socket.IO.

1.4 Objectives

- Reduce manual order errors.
- Enable customers to order directly from their tables.
- Validate availability, quantities and prices on the server.
- Provide real-time kitchen and customer status updates.
- Centralize restaurant information for management.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

- Development computer capable of running Node.js, Vite and MongoDB.
- Customer smartphone with camera/QR scanner and modern browser.
- Network connectivity for frontend/backend communication.

2.2 Software Requirements

- Node.js and npm
- MongoDB
- Chrome, Edge or Firefox
- Visual Studio Code or equivalent editor
- Git/GitHub

2.3 Functional Requirements

- Customer: QR validation, menu, cart, checkout and tracking.
- Chef: receive orders and update status.
- Manager: manage menu/categories/tables and monitor KPIs.
- Backend: REST API, authentication, authorization and data persistence.
- Real-time: new-order and status-update events through Socket.IO.

CHAPTER 3: TECHNOLOGY STACK

Technology	Purpose
React 19 + Vite	Frontend SPA and build tooling
React Router	Customer, Chef and Manager navigation
Axios	HTTP client for REST APIs
Node.js + Express.js	Backend REST API
MongoDB + Mongoose	Persistent data and models
Socket.IO	Real-time kitchen/order updates
JWT + HTTP-only cookies	Staff authentication
Argon2	Password hashing
Helmet / CORS / Rate Limit	Server security controls
QRCode React	Table QR-code generation

3.1 Frontend

React 19 with Vite provides the single-page application. React Router manages navigation, Axios handles HTTP calls, and contexts maintain authentication, cart/table state, and Socket.IO connectivity.

3.2 Backend

Node.js with Express.js provides REST APIs for authentication, tables, menu data, and orders. Controllers perform business validation while middleware provides authentication, role authorization, error handling and security.

3.3 Database

MongoDB stores Users, Tables, Categories, MenuItems and Orders through Mongoose schemas.

CHAPTER 4: SYSTEM ARCHITECTURE

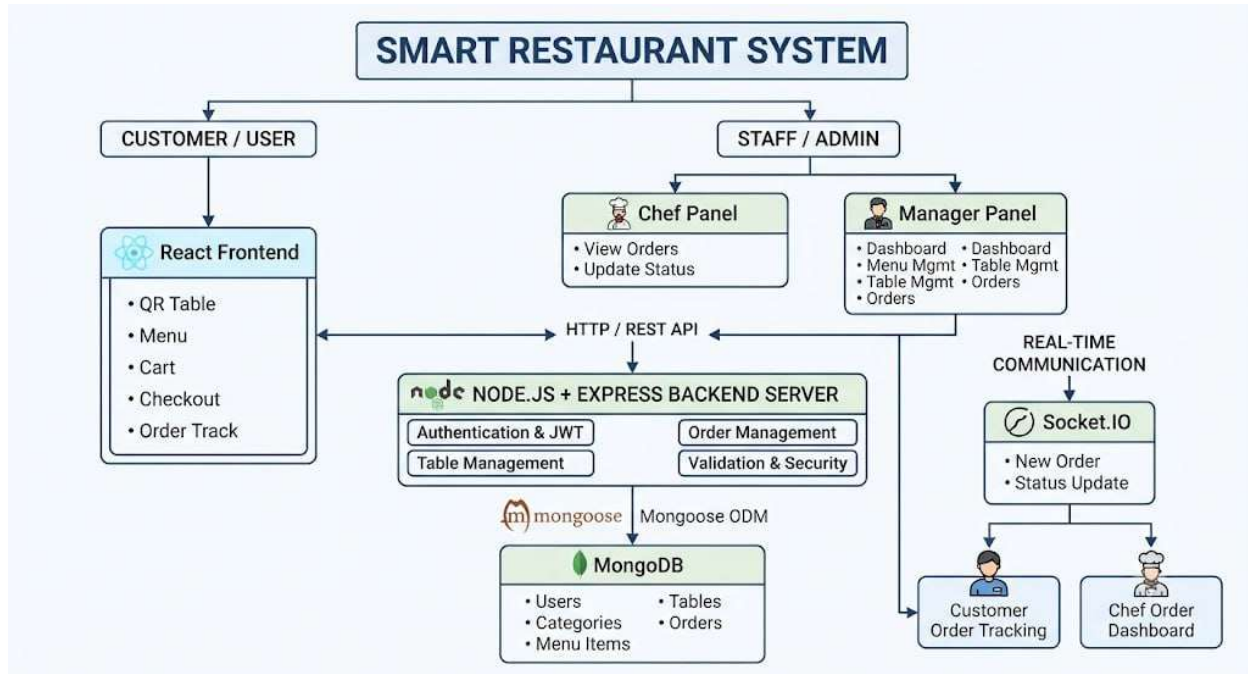


Figure 4.1: High-level architecture from the submitted project presentation.

4.1 Presentation Layer

The React frontend provides separate customer, chef and manager experiences. Customers use the menu and checkout flow, while staff use protected dashboards.

4.2 Application Layer

Express routes are mounted under `/api/auth`, `/api/tables`, `/api/menu` and `/api/orders`. Authentication and role middleware protect staff functions.

4.3 Data Layer

Mongoose models provide structured persistence in MongoDB. Order items store name and price snapshots in addition to the menu item reference.

4.4 Real-Time Layer

Socket.IO provides a kitchen room for new orders and order-specific rooms for customer tracking.

CHAPTER 5: DATABASE DESIGN

Collection	Important Fields
User	name, email, passwordHash, role
Table	tableNumber, capacity, status, qrToken
Category	name, description, isActive
MenuItem	name, description, price, image, category, availability
Order	orderNumber, table, items, subtotal, tax, total, status, timeline

5.1 Relationships

- Orders reference Tables.
- MenuItems reference Categories.
- Orders contain embedded item snapshots and reference the original MenuItems.
- Users have MANAGER or CHEF roles.

5.2 Order State Model

The supported states are PENDING, PREPARING, READY, DELIVERED and CANCELLED. The order controller prevents backward movement for non-cancelled transitions and records timeline timestamps.

5.3 Data Integrity

Schema constraints include required fields, unique identifiers such as tableNumber and qrToken, minimum numeric values, enumerated roles/statuses, and timestamp fields.

CHAPTER 6: IMPLEMENTATION

6.1 Customer Module

- ValidateTable reads the QR token and calls table validation.
- Menu displays categories and active/available items.
- CartContext maintains selected items and quantities.
- Checkout submits only menu item IDs and quantities.
- Backend revalidates table, availability, quantity and price.

6.2 Chef Module

- Chef dashboard receives order:new events.
- Orders are managed through the kitchen status workflow.
- Protected routes allow Chef/Manager status updates.
- Socket.IO communicates status changes to order tracking.

6.3 Manager Module

- Manager dashboard calculates today's revenue from delivered orders.
- Dashboard shows today's orders, active orders and completed orders.
- Menu/category management and table/QR management are available to the manager.

```
54  
55 // Mount Routers  
56 app.use('/api/auth', authRoutes);  
57 app.use('/api/tables', tableRoutes);  
58 app.use('/api/menu', menuRoutes);  
59 app.use('/api/orders', orderRoutes);  
70
```

Figure 6.1: Backend API route mounting evidence.

CHAPTER 7: FEATURES

Feature	Description
QR Table Access	Unique QR token validates a restaurant table before ordering.
Digital Menu	Customers browse categories and menu items with price and availability.
Cart & Checkout	Customers choose quantities and place orders.
Server-Side Pricing	Backend recalculates subtotal, 5% tax and total.
Kitchen Display	Chef receives new orders and moves them through status stages.
Live Tracking	Customers receive order status updates through Socket.IO.
Manager KPIs	Revenue, order count, active and completed order indicators.
Menu & Table Management	Managers maintain menu items, categories, tables and QR codes.
Security	JWT, HTTP-only cookies, Argon2, Helmet, CORS and rate limiting.

CHAPTER 8: TESTING & VALIDATION

The following representative test scenarios are derived directly from the implemented validation and controller logic.

Test Case	Expected Result
Invalid/inactive QR token	Reject validation; menu not opened
Unavailable menu item	Reject order item
Quantity <1 or >50	Reject order
Client price manipulation	Backend ignores client price and reads DB price
Missing table/items	Return validation error
Unauthorized staff route	Require JWT and correct role
Invalid order status	Reject unsupported status
Backward status transition	Reject backward non-cancelled transition
New order	Emit order:new to kitchen room

8.1 Security Validation

- Login attempts are rate-limited.
- Order creation is rate-limited.
- JWT authentication protects staff routes.
- Role middleware restricts Chef/Manager functionality.
- Helmet and CORS are configured.
- Argon2 is used for password hashing.

CHAPTER 9: DEPLOYMENT

9.1 Development Setup

- Install backend dependencies and configure environment variables.
- Start MongoDB.
- Run the Express backend; default PORT is 5000.
- Install frontend dependencies and run Vite; default development client is 5173.
- Open the frontend in a modern browser.

9.2 Environment Configuration

- MongoDB connection details.
- JWT signing secret.
- Server PORT.
- CLIENT_URL/CORS origin.

9.3 Production Considerations

- Use HTTPS and managed MongoDB.
- Restrict CORS to trusted origins.
- Authenticate Socket.IO room joins.
- Use secure secret management.
- Add automated regression and end-to-end testing.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Challenges

- Keeping customer, chef and manager views synchronized.
- Making server-side prices authoritative.
- Synchronizing REST order changes with Socket.IO clients.
- Protecting staff functionality using role authorization.
- Maintaining a table-specific ordering flow.

10.2 Current Limitations

- Online payment is not integrated; the checkout notes payment at the table or counter.
- Customer order retrieval is simplified and should use stronger table/session authorization in production.
- Socket.IO room joining should be authenticated in production.
- The backend package does not include an automated test script.

CHAPTER 11: FUTURE ENHANCEMENTS

- Online payments and digital receipts.
- Kitchen printer/display integration and push notifications.
- Stronger customer session/table authorization.
- Authenticated Socket.IO room membership.
- Advanced sales analytics and reporting.
- Multi-branch support.
- Inventory integration.
- Customer feedback and order history.
- Automated unit, API and end-to-end testing.

CHAPTER 12: CONCLUSION

The Smart Restaurant Ordering & Kitchen Display System digitizes restaurant ordering and kitchen coordination using QR-based tables, a digital menu, MongoDB, REST APIs and real-time Socket.IO communication.

The completed architecture connects customer ordering, chef processing and manager administration. Server-side validation protects order pricing and availability, while authentication and role authorization protect staff functionality.

The system provides a foundation for a contactless restaurant workflow and can be extended with payments, stronger authorization, printer/display integration, analytics and automated testing.

12.1 Skills Demonstrated

- React/Vite development
- Node.js/Express REST APIs
- MongoDB/Mongoose modeling
- JWT and role authorization
- Socket.IO real-time communication
- QR-based table workflows
- Full-stack integration and debugging

CHAPTER 13: REFERENCES

- React Documentation — <https://react.dev/>
- Vite Documentation — <https://vite.dev/>
- Node.js Documentation — <https://nodejs.org/>
- Express.js Documentation — <https://expressjs.com/>
- MongoDB Documentation — <https://www.mongodb.com/docs/>
- Mongoose Documentation — <https://mongoosejs.com/>
- Socket.IO Documentation — <https://socket.io/docs/>
- JSON Web Token — <https://jwt.io/>
- Argon2 / node-argon2 package documentation
- Submitted Smart Restaurant Ordering & Kitchen Display System presentation and project source code

CHAPTER 14: APPENDICES

Appendix A: Implementation Screenshots

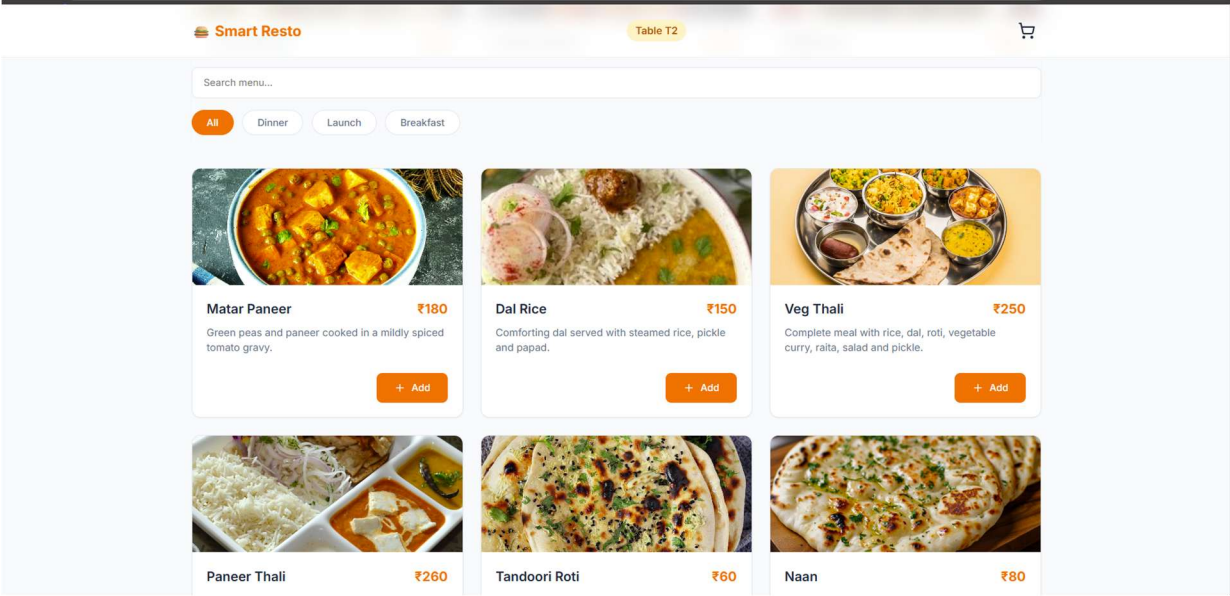


Figure A.1: Customer digital menu.

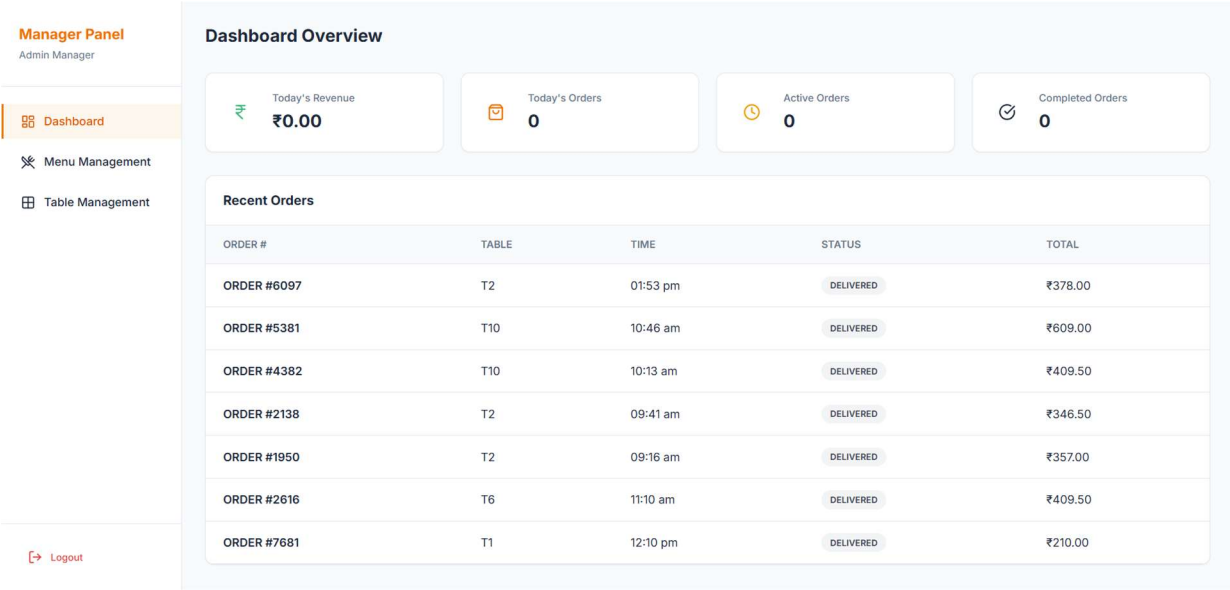


Figure A.2: Manager dashboard.

Appendix B: API Reference Summary

Resource	Endpoints
Auth	POST /api/auth/login POST /api/auth/register GET /api/auth/me POST /api/auth/logout
Tables	GET /api/tables/validate/:qrToken GET/POST /api/tables GET/PUT /api/tables/:id
Menu	GET /api/menu GET /api/menu/categories POST /api/menu PUT /api/menu/:id
Orders	POST /api/orders GET /api/orders/:id GET /api/orders PUT /api/orders/:id/status

Appendix C: Project Identification

- Team - 25 | Section - 12 | CSIT
- Hitesh Suthar - 2520090078
- V. Nikith Chowdary - 2520090219
- B. Vasudeva - 2520090136
- Guide - Dr. Khajashareef
- Project Domain - Full-Stack Web Application
- Platform - Responsive Web
- Modules - Customer Ordering | Chef Kitchen Dashboard | Manager Dashboard

This documentation is based on the submitted project presentation, project source code, implementation screenshots, and the supplied final-documentation structure. Production-only capabilities are not presented as completed features unless supported by the project materials.